You can run Assignment 4: Multithreading and Resource Utilization in Ubuntu Terminal using GCC. I'll explain each question separately, including commands, code, output and execution.

Assignment 4 – Multithreading and Resource Utilization

Step 1: Open Ubuntu Terminal

Press `Ctrl + Alt + T`.

First, install GCC and the required tools:

```
sudo apt update
sudo apt install build-essential time htop -y
```

Check GCC:

```
gcc --version
```

Create a folder for Assignment 4:

```
mkdir Assignment4
cd Assignment4
```

Q1. Create three POSIX threads

Aim: Create three threads using `pthread_create()`. Each thread prints its thread ID. Main waits using `pthread_join()`.

Step 1: Create the C file

```
nano threads.c
```

Step 2: Write this code

```
#include <stdio.h>
#include <pthread.h>

void *task(void *arg)
{
    int id = *(int *)arg;

    printf("Thread %d is running. Thread ID: %lu\n",
           id, (unsigned long)pthread_self());

    return NULL;
}

int main()
{
    pthread_t t[3];
    int id[] = {1, 2, 3};

    for(int i = 0; i < 3; i++)
    {
        pthread_create(&t[i], NULL, task, &id[i]);
    }

    for(int i = 0; i < 3; i++)
    {
        pthread_join(t[i], NULL);
    }

    printf("All threads completed.\n");

    return 0;
}
```

Step 3: Save the file

- Press `Ctrl + O`

- Press `Enter`
- Press `Ctrl + X`

Step 4: Compile

```
gcc threads.c -o threads -pthread
```

Step 5: Run

```
./threads
```

Expected output:

```
Thread 1 is running. Thread ID: ...
Thread 2 is running. Thread ID: ...
Thread 3 is running. Thread ID: ...
All threads completed.
```

Thread execution order may change.

Q2. Calculate array sum using N threads

Aim: Divide an array among N threads and calculate the total sum using 1, 2, 4 and 8 threads.

Step 1: Create the file

```
nano sum_threads.c
```

Step 2: Write this code

```c
#include <stdio.h>
#include <pthread.h>

#define SIZE 1000000
#define MAX_THREADS 8

int arr[SIZE];
long long partial[MAX_THREADS];

typedef struct
{
    int id;
    int start;
    int end;
} Data;

void *sum_part(void *arg)
{
    Data *d = (Data *)arg;
    long long sum = 0;

    for(int i = d->start; i < d->end; i++)
    {
        sum += arr[i];
    }

    partial[d->id] = sum;

    return NULL;
}

int main()
{
    pthread_t t[MAX_THREADS];
    Data d[MAX_THREADS];

    int nthreads;
    long long total = 0;

    printf("Enter threads (1, 2, 4 or 8): ");
```

```
    scanf("%d", &nthreads);

    if(nthreads != 1 && nthreads != 2 &&
       nthreads != 4 && nthreads != 8)
    {
        printf("Invalid thread count.\n");
        return 1;
    }

    for(int i = 0; i < SIZE; i++)
    {
        arr[i] = 1;
    }

    for(int i = 0; i < nthreads; i++)
    {
        d[i].id = i;
        d[i].start = i * SIZE / nthreads;
        d[i].end = (i + 1) * SIZE / nthreads;

        pthread_create(&t[i], NULL, sum_part, &d[i]);
    }

    for(int i = 0; i < nthreads; i++)
    {
        pthread_join(t[i], NULL);
    }

    for(int i = 0; i < nthreads; i++)
    {
        total += partial[i];
    }

    printf("Total sum = %lld\n", total);

    return 0;
}
```

Step 3: Save the file

Press `Ctrl + O` → `Enter` → `Ctrl + X`.

Step 4: Compile

```
gcc sum_threads.c -o sum_threads -pthread
```

Step 5: Run with 1 thread

```
./sum_threads
```

Enter:

```
1
```

Output:

```
Enter threads (1, 2, 4 or 8): 1
Total sum = 1000000
```

Step 6: Run with 2 threads

```
./sum_threads
```

Enter 2.

Step 7: Run with 4 threads

```
./sum_threads
```

Enter 4.

Step 8: Run with 8 threads

```
./sum_threads
```

Enter 8.

Expected result

All four executions should display:

```
Total sum = 1000000
```

Q3. Measure execution time and resource usage

Aim: Measure execution time, memory usage and CPU usage for 1, 2, 4 and 8 threads.

Step 1: Measure execution time

Run:

```
/usr/bin/time -v ./sum_threads
```

Enter 1.

Note the following values:

- User time
- System time
- Elapsed time
- Maximum resident set size

Repeat the command with 2, 4 and 8 threads.

Step 2: Check CPU and memory usage

Open another Ubuntu Terminal using `Ctrl + Alt + T`.

Run:

```
top
```

Or:

```
htop
```

If `htop` is not installed:

```
sudo apt install htop -y
```

Press `q` to exit.

Step 3: Prepare the observation table

Record your actual measurements.

Threads	Execution time	Memory usage	CPU usage
1	Measure	Measure	Measure
2	Measure	Measure	Measure
4	Measure	Measure	Measure
8	Measure	Measure	Measure

Note: Do not copy imaginary measurements. Record the values from your Ubuntu computer.

Final practical file list

After completing all three questions, check your files:

```
ls
```

Expected:

```
threads.c
threads
sum_threads.c
sum_threads
```

Important viva questions

- What is a thread? A thread is the smallest unit of execution within a process.
- What is pthread? POSIX thread library used for multithreading in C.
- What is pthread_create()? It creates a new thread.
- What is pthread_join()? It waits for a thread to finish.
- What is parallel processing? Executing multiple tasks simultaneously using multiple CPU cores.
- What is resource utilization? Measurement of CPU and memory usage.
- Why use multiple threads? To improve performance and execute tasks concurrently.

Start practical now

First, execute these commands in Ubuntu:

```
sudo apt install build-essential time htop -y
mkdir Assignment4
cd Assignment4
nano threads.c
```

Start with Q1. Once you get the output, proceed to Q2 and Q3.